



Jasmin Code

Question 1.

Write Jasmin code for the following C expressions:

- $a + b$
- $a + b * 4$
- $(a + 32) \% (b - 100)$

where a and b are integer variables whose indexes are 0 and 1, respectively.



Question 2.

Give the AST classes as follows,

```
class IntLiteral(Exp):# value: int
class FloatLiteral(Exp): #value: float
```

Write methods `visitIntLiteral(self,ast,o)` and `visitFloatLiteral(self,ast,o)` of class `CodeGeneration` to generate the code for an integer and float literal, respectively? The class `CodeGeneration` has field `emit` keeping an object of `Emitter` Object `Frame` is kept in field `frame` of the argument passed to parameter `o` of `visitIntLiteral`. The methods `visitIntLiteral` and `visitFloatLiteral` must return a pair of jasmin code of loading an int/float constant into operand stack and the type of the constant (one object of a subclass of class `Type`)

Remind the related method in the class `Emitter`:

```
emitPUSHICONST(self , in:Union[int|str] , frame:Frame)
emitPUSHFCONST(self , in:str , frame)
emitPUSHCONST(self , in:str , typ: Type,frame)
```



Question 3.

Give the AST classes as follows,

```
class BinExp(Exp):# op:str [+,-,*,/],
                  left:Exp,right:Exp
```

Write methods `visitBinExp(self,ast,o)` of class `CodeGeneration` to generate the code for a binary expression? The class `CodeGeneration` has field `emit` keeping an object of `Emitter` Object `Frame` is kept in field `frame` of the argument passed to parameter `o` of `visitBinExp`. The methods `visitBinExp` must return a pair of jasmin code of evaluation the binary expression and the type of the expression result.

Remind the related method in the class `Emitter`:

```
emitADDOP(self , lex: str , in_:Type, frame:Frame) # +, -
emitMULOP(self , lex: str , in_:Type, frame:Frame) # * , /
emitDIV(self , frame:Frame) # integer division
emitREOP(self , op:str , in_:Type, frame:Frame)
emitI2F(self ,frame:Frame)
```



Question 4.

Write Jasmin code for the following C statements:

- $c = a > b$;
- $c = (a \geq b) \ \&\& \ (a < 100)$;

where a, b and c are integer variables whose indexes are 0, 1 and 2, respectively.

**Question 5.**

Give the AST classes as follows,

```
class Id(Exp):# name:str
```

Write methods `visitId(self,ast,o)` of class `CodeGeneration` to generate the code for an identifier?

- The class `CodeGeneration` has field `emit` keeping an object of `Emitter`
- Object is passed to the parameter `o` of `visitId` has 3 fields:
 - Field `frame` keeps Object `Frame`.
 - Field `sym` of the argument keeps a list of `Symbol` which has three fields: `name` (str type), `mtype` (Type type) and `value` (Val type). The Val type has two concrete classes: `Index` with field `value` in int type and `CName` with field `value` in str type. An `Index` object keeps the index of the variable while a `CName` keeps the name of the class name (used for global variable). The first element of `sym` contains the identifier which belongs to the innermost referencing environment while the last element of `sym` contains one in the outermost referencing environment (global).
 - Field `isLeft` in boolean type indicates the identifier in the left (`isLeft` true) or in the right (`isLeft` false).

The methods `visitId` must return a pair of jasmin code to read or write value of the identifier and the type of the identifier (one object of a subclass of class `Type`).

Remind the related method in the class `Emitter`:

```
emitREADVAR(self , lex: str , in_: Type, idx: int , frame: Frame)
emitWRITEVAR(self , lex: str , in_: Type, idx: int , frame: Frame)
emitGETSTATIC(self , lex: str , in_: Type, frame: Frame)
emitPUTSTATIC(self , lex: str , in_: Type, frame: Frame)
emitI2F(self , frame: Frame)
```



Question 6.

Write Jasmin code generated by your code generation for the following C statements:

- if ($a > b$) $a = 1$; else $a = 2$;
- while ($a < 10$) $a = a + 1$;
- for ($a = 1; a < 10; a++$) $b = b + 1$;

where a and b are integer variables whose indexes are 0 and 1, respectively.



Question 7.

Give the AST classes as follows,

```
class If(Stmt):# cond:Exp, thenStmt:Stmt, elseStmt:Stmt
```

Write methods `visitIf(self, ast, o)` of class `CodeGeneration` to generate the code for an if statement?

- The class `CodeGeneration` has field `emit` keeping an object of `Emitter`
- Object is passed to the parameter `o` of `visitIf` has 2 fields:
 - Field `frame` keeps Object `Frame`.
 - Field `sym` of the argument keeps a list of `Symbol` which has three fields: `name` (str type), `mtype` (Type type) and `value` (Val type). The Val type has two concrete classes: `Index` with field `value` in int type and `CName` with field `value` in str type. An `Index` object keeps the index of the variable while a `CName` keeps the name of the class name (used for global variable). The first element of `sym` contains the identifier which belongs to the innermost referencing environment while the last element of `sym` contains one in the outermost referencing environment (global).

When visiting the expression of the if statement, object `Access` must be passed to parameter `o` where `Access` has 3 fields:

- Fields `frame` and `sym` are similar to the object passed to parameter `o`.
- Field `isLeft` in boolean type indicates the identifier in the left (`isLeft` true) or in the right (`isLeft` false).

The methods `visitIf` must print out the code of the if statement (use method `printout` of `Emitter`).

Remind the related methods in the class `Emitter`:

```
emitGOTO(self, label: int, frame: Frame)
emitLABEL(self, label: int, frame: Frame)
emitIFFALSE(self, label:int, frame: Frame)
emitIFTRUE(self, label:int, frame: Frame)
printout(self, val:str)
```

Remind the related methods in the class `Frame`:

```
getNewLabel(self)
```



Question 8.

Give the AST classes as follows,

```
class While(Stmt):# cond:Exp, body:Stmt
```

Write methods `visitWhile(self, ast, o)` of class `CodeGeneration` to generate the code for a while statement?

- The class `CodeGeneration` has field `emit` keeping an object of `Emitter`
- Object is passed to the parameter `o` of `visitIf` has 2 fields:
 - Field `frame` keeps Object `Frame`.
 - Field `sym` of the argument keeps a list of `Symbol` which has three fields: `name` (str type), `mtype` (Type type) and `value` (Val type). The Val type has two concrete classes: `Index` with field `value` in int type and `CName` with field `value` in str type. An `Index` object keeps the index of the variable while a `CName` keeps the name of the class name (used for global variable). The first element of `sym` contains the identifier which belongs to the innermost referencing environment while the last element of `sym` contains one in the outermost referencing environment (global).

When visiting the expression of the while statement, object `Access` must be passed to parameter `o` where `Access` has 3 fields:

- Fields `frame` and `sym` are similar to the object passed to parameter `o`.
- Field `isLeft` in boolean type indicates the identifier in the left (`isLeft` true) or in the right (`isLeft` false).

The methods `visitWhile` must print out the code of the if statement (use method `printout` of `Emitter`).

Remind the related methods in the class `Emitter`:

```
emitGOTO(self, label: int, frame: Frame)
emitLABEL(self, label: int, frame: Frame)
emitIFFALSE(self, label:int, frame: Frame)
emitIFTRUE(self, label:int, frame: Frame)
printout(self, val:str)
```

Remind the related methods in the class `Frame`:

```
getNewLabel(self)
enterLoop(self)
exitLoop(self)
getBreakLabel(self)
getContinueLabel(self)
```




Question 9.

Write Jasmin code for the following C statements:

- $a[0] = a[0] + 1;$
- $b[0][0] = b[0][1] + 1;$

where a and b are one-dimension and two-dimension integer array, respectively.



Question 10.

Give the AST classes as follows,

```
class VarDecl(Decl):# name:str , typ: Type
```

Write methods `visitVarDecl(self,ast,o)` of class `CodeGeneration` to generate the code for a variable declaration?

- The class `CodeGeneration` has field `emit` keeping an object of `Emitter`
- Field `frame` of the argument passed to parameter `o` of `visitVarDecl` contains object `Frame` for a local/parameter declaration and `None` for a global declaration.
- The method `visitVarDecl` must print out a directive declaration in jasmin code (use method `printout(str)` of `Emitter`) and returns an object of `Symbol` which has field `name` in `str` type, `mtype` in `Type` type and `val` in `Val` type. The `Val` type has 2 concrete classes: `Index` with field value in `int` type (contain index of variable) and `CName` with field value in `str` type (contains name of the class which can get from `self.className`)

Remind the related methods in the class `Emitter`:

```
printout(cod: str)
emitATTRIBUTE(self,lex:str,in_:Type,isFinal:bool)
emitVAR(self,idx:int , lex: str , in_: Type, start: int , end:int)
```

and the related methods in the class `Frame`:

```
enterScope()
exitScope()
getStartLabel()
getEndLabel()
```